Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović[1,5], Christian Blum[2], Aleksandar Kartelj[3], Sašo Džeroski[4], and Žiga Zebec[5]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iiia.csic.es
[3] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[4] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[5] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. This study addresses the Variable Gapped Longest Common Subsequence Problem (VGLCSP), a variant of the classical longest common subsequence problem with additional gap constraints and applications in sequence alignment and time-series analysis. While the two-sequence version has been widely studied using dynamic programming, the generalized multi-sequence form is usually solved with beam search-based heuristics, whose hand-crafted designs often lack robustness.
To overcome this limitation, we propose a learning-based approach for automatically designing more effective data-driven heuristics. The heuristics are represented by a neural network with predefined architecture, whose weights are optimized by a genetic algorithm within a neuro-evolutionary framework. The learning process alternates between weight optimization and evaluation within an iterative multi-source beam search procedure, a state-of-the-art method for the problem. Rather than constructing solutions directly, the neural network learns to guide the search process, producing a neuro-evolved heuristic. We further introduce an ensemble heuristic that combines the learned and the best-performing hand-crafted heuristic. Integrated into the iterative multi-source beam search framework, the resulting hybrid approach outperforms existing methods on both synthetic benchmark instances and newly introduced real-world instances with data-driven gap constraints.

Keywords: Beam search; Neural networks; Neuro-Evolved heuristics; Gap constraints

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [5] is a fundamental combinatorial optimization problem with numerous applications in computational

biology, particularly in the analysis of molecular sequences. Given a set of input sequences $S = \{s_1, \ldots, s_m\}$ over an alphabet Σ, the objective is to determine the longest subsequence common to all sequences $s_i \in S$. Over the past decades, several practically motivated variants of LCSP have been proposed by incorporating additional structural or biological constraints, including the constrained, arc-preserving, and repetition-free variants [10, 12].

In this work, we study the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP), introduced in [16], theoretically analyzed in [1], and recently evaluated experimentally in [4]. This variant extends the classical LCSP by introducing flexible gap constraints between consecutive symbols in the subsequence, allowing these gaps to vary along the input sequences.

Formally, for each sequence $s_i \in S$, a gap function $G_{s_i} : \{1, \ldots, |s_i|\} \to \mathbb{N}$ specifies the maximum allowed distance between consecutive matched positions. If a solution s appears at positions $i_i^1 < \cdots < i_i^{|s|}$ in s_i, the gap constraints are satisfied if

$$i_i^x - i_i^{x-1} \leq G_{s_i}[i_i^x] + 1, \quad \forall x = 2, \ldots, |s|.$$

A subsequence is feasible if these constraints hold for all sequences $s_i \in S$. The objective is to find the longest feasible subsequence. The VGLCSP provides a flexible and realistic model for sequence alignment, particularly relevant for DNA and protein analysis with variable structural distances, as well as for time-series analysis where events must occur within specified temporal delays [11].

While several exact dynamic programming approaches exist for the two-sequence case ($m = 2$) [15], the problem becomes computationally intractable for larger m; in fact, it is NP-hard in the general case. To date, no efficient exact methods are known for arbitrary m. The only available scalable approach is an iterative multi-source beam search (IMSBS) [4], which explores a state-space representation of the problem. The method iteratively refines candidate root nodes and extends them through feasible symbol additions. Despite its scalability, IMSBS relies on hand-crafted heuristics, which tend to degrade in performance as m increases or the alphabet size $|\Sigma|$ decreases [5]. The presence of gap constraints further exacerbates this issue.

In addition, existing studies are limited to synthetic benchmark instances, leaving a gap in evaluating methods on realistic data.

To address these limitations, we propose a learning-enhanced beam search framework for VGLCSP. Our approach integrates learned and hand-crafted heuristics into a unified search strategy. The learned heuristic is modeled by a multi-layer neural network that scores candidate states—that is, feasible extensions of partial solutions—during the search. The network parameters are optimized through a genetic algorithm in a neuro-evolutionary framework, where each candidate model is evaluated within the IMSBS. Importantly, the neural model does not directly construct solutions but learns to guide the search process.

Furthermore, we design an ensemble heuristic that combines the learned heuristic with the best-performing hand-crafted heuristic, yielding a more robust guidance mechanism. This hybrid approach is inspired by recent advances

in learning-guided combinatorial optimization [6, 18, 22], but differs in explicitly leveraging complementary heuristic signals within an ensemble.

Finally, to address the lack of realistic benchmarks, we introduce new problem instances derived from biologically motivated datasets, augmented with data-driven gap constraints.

Contributions. The main contributions of this work are:

- A neuro-evolutionary framework designed for learning heuristics (resp. heuristic guidance) in the context of the VGLCSP.
- An ensemble heuristic combining learned and hand-crafted guidance.
- An enhanced IMSBS algorithm integrating the proposed ensemble heuristics.
- A new set of realistic benchmark instances with data-driven gap constraints.

Preliminaries. Let $|s|$ denote the length of a sequence s, and let $s[i]$ refers to the character of s at position i, where indexing starts at $i = 1$. The substring of s that begins at position i and ends at position j is denoted by $s[i, j]$. If $i = j$, this corresponds to the single character $s[i]$; if $j < i$, then $s[i, j] = \varepsilon$, where ε denotes the empty string. Given two sequences s_1 and s_2, by $s_1 \cdot s_2$ we denote the sequence obtained by concatenating the given two sequences. The number of occurrences of $a \in \Sigma$ in s is denoted by $|s|_a$. We denote by $S = \{s_1, \ldots, s_m\}$ the set of input sequences, $m \in \mathbb{N}$ the number of input sequences (and, correspondingly, the number of gap constraints), and n denotes the length of the longest sequence in S. Given a positional vector $\mathbf{p}^L \in \mathbb{N}^m$, where $1 \leq p_i^L \leq s_i$ for all $s_i \in S$, the subproblem related to these positions is given by $S[\mathbf{p}^L] = \{s_i[\mathbf{p}_i^L, |s_i|] \mid i = 1, \ldots, m\}$.

The paper is organized as follows. In Section 2, the iterative multi-source beam search approach from the literature is roughly presented. Section 3 provides the main details on the process of learning heuristic guidance and the design of an ensemble heuristic estimator for the tackled problem. Section 4 reports a rigorous experimental analysis. Finally, Section 5 concludes the paper, highlighting several future research directions.

2 Iterative Multi-Source Beam Search for the VGLCSP

The first scalable approach for the VGLCSP with an arbitrary number of sequences (m) was introduced in [4] as an *Iterative Multi-Source Beam Search* (IMSBS). In this section, we summarize its main components; interested readers are referred to the original paper for full details.

IMSBS is based on a *state graph* representation of the VGLCSP. Each node represents a partial feasible subsequence, characterized by a vector $\mathbf{p}^{L,v}$ of positions defining a subproblem and the length l^v of the constructed subsequence. Formally, a state $v = (\mathbf{p}^{L,v}, l^v)$ is induced by a partial solution s^v if: (i) $p_i^{L,v} - 1$ is the smallest index such that s^v is a subsequence of the prefix $s_i[1, p_i^{L,v} - 1]$; (ii) $l^v = |s^v|$; and (iii) all gap constraints G_{s_i} are satisfied by s^v.

A directed arc $\alpha = (v_1, v_2)$ labeled by $lett(\alpha) = a \in \Sigma$ exists if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$. To expand a state v, all letters occurring in the suffixes $S[\mathbf{p}^{L,v}]$ are considered. For each such letter a, the smallest feasible positions $p_i^{L,a} \geq p_i^{L,v}$ are identified such that the gap constraints are satisfied, i.e., $p_i^{L,a} - p_i^{L,v} \leq G_{s_i}(p_i^{L,a})$. The resulting child state (if not dominated) is $w = (\mathbf{p}^{L,a} + \mathbf{1}, l^v + 1)$. Terminal states correspond to non-extendable feasible subsequences.

Unlike the classical LCSP, the VGLCSP admits multiple (potentially exponentially many) root nodes, since starting positions are unconstrained. This may lead to disconnected regions in the state graph. For instance, consider $s_1 = \texttt{ATGGAAAA}$ and $s_2 = \texttt{ATCCAAAA}$ with $G_{s_1} = G_{s_2} = 1$. The root node $((1,1),0)$ cannot reach states with position vector $(5,5)$, thus missing the optimal subsequence `AAAA`. This phenomenon is illustrated in Fig. 1, where the state graph decomposes into disconnected components.

This observation motivates the IMSBS framework, which operates over a dynamically generated set of root nodes explored iteratively. At each iteration, beam search (BS) is applied to selected root nodes. BS is a breadth-first heuristic search strategy [21] that expands a limited number of nodes per level, controlled by the beam width β, and guided by a heuristic function h.

In the original IMSBS, node selection relies on an upper-bound heuristic defined as

$$\texttt{UB}(v) = l^v + \sum_{a \in \Sigma} \min_{i=1,\ldots,m} |s_i[p_i^{L,v}, |s_i|]|_a,$$

which estimates the maximum achievable extension from state v. The heuristic evaluates remaining occurrences of each letter $a \in \Sigma$ in each suffix sequence of the related subproblem and aggregates these values to derive an upper bound on the achievable subsequence length.

Despite its scalability, IMSBS relies on hand-crafted heuristics whose effectiveness degrades in challenging settings (e.g., large m or small alphabets), especially under gap constraints. To resolve the issue, we will develop a learning-based methodology that enhances the search process of the IMSBS through the integrated learned heuristic guidance.

In the subsequent part of the paper, we refer to the algorithm as $\texttt{IMSBS}(\mathcal{I}, \mathcal{C})$, returning an approximate solution s_{imsbs} for an instance $\mathcal{I}$ under the parameter configuration $\mathcal{C}$.

3 Learning to Guide the Search

Neural networks (NNs) are well-established machine learning models [9, 3] that have demonstrated strong performance across a wide range of domains. They range from simple multi-layer perceptrons (MLPs) to more complex convolutional and recurrent architectures [13]. In this work, we focus on fully connected multi-layer perceptrons to design an alternative, data-driven heuristic for guiding the IMSBS procedure [4].

The key challenge lies in parameterizing the neural model such that it provides meaningful guidance during the search. Specifically, the network is trained

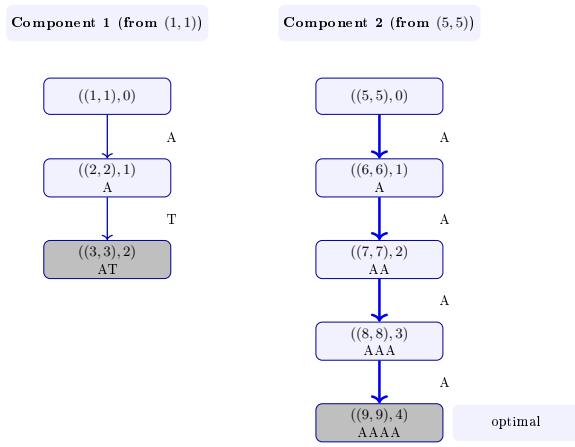

Fig. 1. State graph for $s_1 = \texttt{ATGGAAAA}$ and $s_2 = \texttt{ATCCAAAA}$ with $G_{s_1} = G_{s_2} = 1$. The state graph consists of two disconnected components. Starting from $(1, 1)$ leads to a suboptimal solution, while starting from $(5, 5)$ yields the optimal subsequence `AAAA`.

to assign a score to each node—that is, to each feasible extension of a current partial solution—based on features extracted from the corresponding partial solution and the associated subproblem.

In standard neural network training, parameters are optimized via forward and backward propagation, typically using gradient-based methods such as gradient descent [2]. However, in our setting—characterized by a combinatorial NP-hard problem—target values are not readily available, especially for large instances. This limits the applicability of supervised learning approaches. Instead, we adopt a population-based metaheuristic approach, where the parameters of NN (as configurations) are evaluated indirectly through their impact on the performance of the IMSBS algorithm.

Predefined NN Architecture. The heuristic is modeled as a multi-layer perceptron consisting of an input layer, several hidden layers with user-defined sizes, and a single output neuron. The input layer receives a vector of numerical features describing the node (partial solution) to be evaluated, while the output represents the estimated promise of producing that state for further consideration in the IMSBS.

For each layer l, forward propagation is defined as

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ denote the weights and biases associated with layer l, and ϕ is a non-linear activation function (e.g., `tanh`, `ReLU`, or `sigmoid`). The final value is used as the heuristic score. All parameters are flattened into a single vector w which represents an individual in the evolutionary process.

Fitness Evaluation. Given a candidate weight vector w, the evaluation proceeds as follows: (i) the weights are loaded into the pre-defined NN architecture

$\mathcal{N}$; (ii) IMSBS is executed on each instance from a training set $\mathcal{T}$, making use of the induced neural heuristic h_w to guide the search (other parameters of IMSBS are fixed, see Section 4); and (iii) the best solution quality obtained per instance is recorded. The fitness of w is defined as the average solution quality over $\mathcal{T}$:

$$\max_{w} \; \mathit{Fitness}(w) = \frac{1}{|\mathcal{T}|} \sum_{I \in \mathcal{T}} f(\texttt{IMSBS}(I, h_w)), \tag{1}$$

where $f(\cdot)$ denotes the length of the best subsequence found. Validation performance is computed analogously on a separate dataset $\mathcal{V}$.

Feature Extraction. To guide the search effectively, each node $v = (\mathbf{p}^v, l^v)$ is mapped to a fixed-length feature vector capturing both the construction progress and the structure of the related subproblem. The design is instance-independent and robust across varying problem sizes.

Let $S = \{s_1, \ldots, s_m\}$ denote the input sequences. The position vector $\mathbf{p}^v$ is first normalized by sequence lengths:

$$F_i^v = \frac{p_i^v}{|s_i|}, \quad i = 1, \ldots, m,$$

ensuring scale invariance. From this normalized vector, we extract four statistics: $\max(\mathbf{F}^v)$, $\min(\mathbf{F}^v)$, $\texttt{mean}(\mathbf{F}^v)$, and $\texttt{std}(\mathbf{F}^v)$, capturing the progress and imbalance across sequences. The current subsequence length l^v is also included.

To reflect future feasibility, we compute gap-related statistics:

$$G^{\min},\ G^{\max},\ G^{\texttt{avg}},\ G^{\texttt{std}},$$

evaluated over the gap values for the remaining positions in all sequences. These features quantify the flexibility of future extensions: larger gaps indicate more freedom, while smaller or highly variable gaps suggest tighter constraints.

Finally, two global features—alphabet size $|\Sigma|$ and the number of sequences m—are included. All features are standardized to zero mean and unit variance prior to training. The resulting 11-dimensional representation provides a compact yet expressive description of each search state.

Optimization of NN Parameters. To optimize the NN parameters, we employ a *Biased Random-Key Genetic Algorithm* (BRKGA), a population-based evolutionary method. Each individual encodes a candidate weight vector w of the NN model, initialized uniformly at random in the interval $[-q, q]$ with $q = 1$. The evolutionary process iteratively refines the population toward high-quality heuristics $h_w = \mathcal{N}(w)$, where $\mathcal{N}$ is a predefined NN architecture, based on the fitness defined in Eq. (1). The overall learning mechanism is summarized in Algorithm 1.

Overall Learning Procedure. The proposed learning framework alternates between evolutionary optimization of neural network parameters and their evaluation within the IMSBS. Importantly, the neural network does not directly construct solutions; instead, it learns to *guide* the search process, placing the approach within the class of neuro-evolutionary hyper-heuristics [7, 14, 8].

Algorithm 1 GA-Based Learning of a Beam Search Heuristic

Input: NN architecture $\mathcal{N}$, IMSBS parameters, BRKGA parameters, training set $\mathcal{T}$, validation set $\mathcal{V}$
Output: optimized weight vector w^*
Initialize population P with random weight vectors
Evaluate all individuals on $\mathcal{T}$
$w^* \leftarrow$ best individual in P
while time limit $t_{\max}$ not exceeded **do**
  Sort P in descending order of *Fitness*
  Initialize new population P'
  Copy top n_{elite} individuals from P to P'
  for $i = 1$ to n_{mutants} **do**
    Generate random individual x
    Evaluate x on $\mathcal{T}$ and add to P'
    Update w^* if improved and validation does not degrade
  end for
  for $i = 1$ to $n_{\text{offspring}}$ **do**
    Select one elite parent and one non-elite parent
    Generate offspring via biased crossover (p_{bias})
    Evaluate offspring on $\mathcal{T}$ and add to P'
    Update w^* if improved and validation does not degrade
  end for
  $P \leftarrow P'$
end while
return w^*

Given a predefined NN architecture $\mathcal{N}$, IMSBS parameters, and BRKGA parameters, the procedure begins by generating an initial population P of candidate weight vectors. Each individual is evaluated on the training set $\mathcal{T}$ using IMSBS guided by the corresponding neural heuristic. The best-performing solution is stored as w^*. In our implementation, IMSBS is executed with parameters $\beta = 500$ and `beam_iters` $= 100$, and the time limit is set to 60 seconds, while the remaining parameters follow [4]. This configuration provides a suitable balance between computational cost and robustness during training.

The evolutionary process proceeds iteratively as follows. The population is first sorted according to fitness. A new population P' is then constructed by preserving the top n_{elite} individuals (elitism), introducing n_{mutants} randomly generated individuals (diversification), and generating $n_{\text{offspring}}$ offspring via biased crossover. In each crossover, one parent is selected from the elite set and the other from the remaining population. Each gene (weight) is inherited from the elite parent with probability $p_{\text{bias}} \geq 0.5$, and from the second parent otherwise. Each newly generated individual is evaluated on $\mathcal{T}$. Whenever an improved solution is found, its performance is additionally assessed on the validation set $\mathcal{V}$. To mitigate overfitting, we adopt an early stopping strategy: if the validation performance deteriorates, the update is discarded.

The process repeats until a time limit $t_{\max}$ is reached (set to 2 CPU hours). The algorithm returns the best weight vector w^*, corresponding to the neural network $\mathcal{N}(w^*)$ that induces the most effective heuristic h_{w^*}.

Our experiments use the following parameters for BRKGA: $n_{\text{pop}} = 20$, $n_{\text{elite}} = 1$, $p_{\text{bias}} = 0.5$, and $n_{\text{mutants}} = 7$. A visualization of the learning process is provided in Fig. 2.

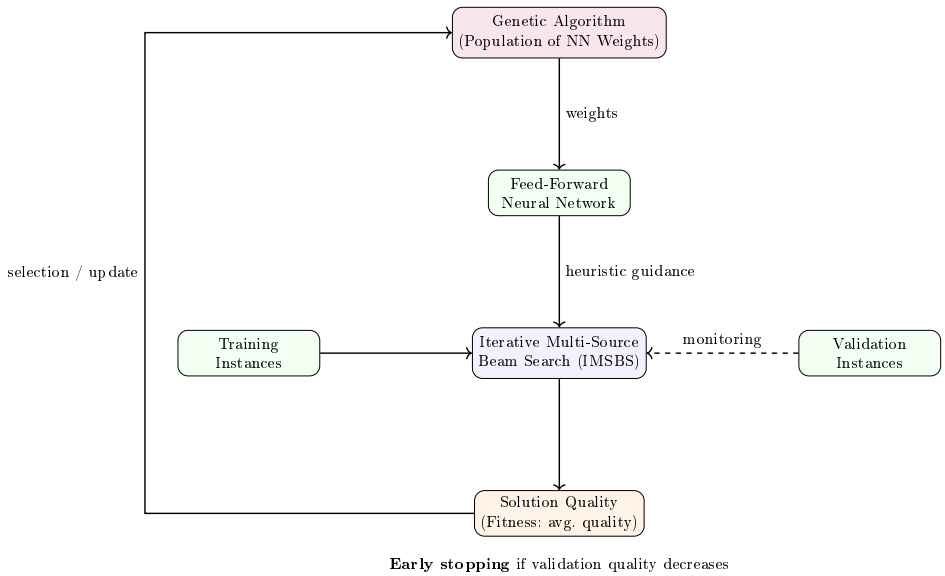

Fig. 2. Enhanced learning framework integrating a Genetic Algorithm with IMSBS. The GA evolves neural network weights, which define a heuristic used within the beam search. Training instances guide optimization, while validation instances monitor generalization. The feedback loop updates the population based on solution quality.

3.1 An ensemble approach

During preliminary experiments, we observed that the learned heuristic generally outperforms the hand-crafted heuristic from the literature. However, this advantage is not consistent across all cases, particularly for instances with small values of m. To improve the results further, we employ a rank-based ensemble strategy that combines the learned heuristic with an analytical hand-crafted upper bound (UB) [4], see Section 2. Specifically, each candidate node in beam search is evaluated using two criteria: a neural score obtained from an FNN $\mathcal{N}(w^*)$, denoted $h_{w^*} = \mathcal{N}(w^*)$, and a problem-specific hand-crafted UB.

Instead of combining the raw scores directly, which may differ in scale and distribution, we adopt a rank aggregation approach. First, candidate nodes are sorted independently according to each criterion, producing two rankings. Each

node is then assigned a rank based on its position in each sorted list. The final ranking score is computed as a weighted combination of the two ranks:

$$r(v) = \alpha \cdot r_{h_{w^*}}(v) + (1 - \alpha) \cdot r_{\mathrm{UB}}(v), \tag{2}$$

Nodes are then sorted in ascending order of $r(v)$, and the top candidates are retained according to the beam width. This rank-based ensemble heuristic provides a robust mechanism for integrating learned and analytical guidance, mitigating scale inconsistencies, and improving search stability.

4 Experimental Evaluation

In this section, we evaluate the performance of two approaches: (i) IMSBS, the baseline state-of-the-art iterative multi-source beam search from [4], and (ii) LIMSBS-ENSEMBLE, an enhanced version of IMSBS guided by the ensemble heuristic defined in Eq. (2).

All methods are implemented in C++ and compiled using the GCC compiler. Experiments are conducted on the VEGA HPC system (IZUM, Maribor, Slovenia), consisting of 960 compute nodes equipped with AMD EPYC 7H12 CPUs at 2.35 GHz. All experiments are executed in a single-threaded setting.

4.1 Benchmark Instances

We consider two types of benchmark sets. Synthetic instances, denoted by RANDOM, are generated in [4]. For each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 instances are available, resulting in a total of 320 problem instances.

Biologically motivated instances are generated to complement synthetic data. These instances involve data-driven gap constraints derived from biological sequence properties. The goal is to model context-dependent positional importance using thermodynamic stability information, inspired by sequence alignment scoring and weighted subsequence models.

We employ a dinucleotide-based structural model derived from nearest-neighbor thermodynamic interactions [19, 20]. The stability of adjacent nucleotides is captured by the function

$$f(s[i], s[i+1]) = -\Delta G(s[i], s[i+1]),$$

where ΔG denotes the free energy associated with the dinucleotide pair. According to [19], the corresponding weights are estimated by the following values:

$$\begin{aligned}\texttt{DINUC_WEIGHT} = \{&\texttt{AA}:1.00, \texttt{TT}:1.00,\\ &\texttt{AT}:0.88, \texttt{TA}:0.58,\\ &\texttt{CA}:1.45, \texttt{TG}:1.45,\\ &\texttt{GT}:1.44, \texttt{AC}:1.44,\\ &\texttt{CT}:1.28, \texttt{AG}:1.28,\\ &\texttt{GA}:1.30, \texttt{TC}:1.30,\\ &\texttt{CG}:2.17, \texttt{GC}:2.24,\\ &\texttt{GG}:1.84, \mathrm{CC}:1.84\}\end{aligned}$$

To ensure consistency with the directional nature of gap constraints, we define a *causal* positional contribution by

$$f_k^s = \begin{cases} f(s[k-1], s[k]), & k > 1, \\ 0, & k = 1. \end{cases}$$

This formulation enforces a left-to-right dependency structure, aligning with the feasibility condition of the VGLCSP.

The positional influence is then computed as:

$$I^s(i) = \sum_{k=\max(1,i-r)}^{i} e^{-\lambda(i-k)} \cdot f_k^s,$$

where r is the influence radius and λ is a decay factor. This captures the cumulative structural context preceding each position.

To normalize influence values, we apply

$$\hat{I}_i^{(j)} = \frac{I_i^{(j)} - \min(I^{(j)})}{\max(I^{(j)}) - \min(I^{(j)}) + \epsilon} \in [0, 1],$$

and derive gap constraints via

$$\mathrm{Gap}_i[j] = w_{\min} + (w_{\max} - w_{\min}) \cdot \hat{I}_i^{(j)}.$$

This mapping assigns larger gaps to structurally stable regions and smaller gaps to less stable ones, reflecting their differing flexibility for matching. For producing instances, we set $w_{\min} = 1$, $w_{\max} = 10$, $r = 10$, and $\lambda = 0.5$.
Moreover, we used the well-known LCS benchmark sets Rat and Virus [5]. For each sequence in these datasets, gap constraints are generated using the above procedure. We restrict attention to instances with $|\Sigma| = 4$, as larger alphabets (e.g., $|\Sigma| = 20$) lead to trivial solutions under gap constraints and are less informative.

The resulting dataset, denoted Real, consists of 20 instances (10 from each benchmark set). All sequences have length 600, while the number of sequences varies from $m = 10$ to $m = 200$.

All benchmark sets are freely available at XX. TODO: Revise

4.2 Tuning Process

The parameters of IMSBS are tuned on the benchmark set RANDOM using `irace` [17], with a total budget of 5000 runs (each limited to 30 minutes). For tuning, one instance from each group is selected, resulting in 32 training instances in total. All remaining (less sensitive) parameters are fixed to the values reported in [4]. The produced configuration for IMSBS is $\beta = 3000$, $h = \text{UB}$, and $\#\texttt{imbs_iters} = 5000$.

For a fair comparison, the proposed LIMSBS-ENSEMBLE inherits the same parameter configuration obtained for IMSBS. The learning process is tuned by exploring different neural network architectures, including two-layer and three-layer multi-layer perceptrons with 5 or 10 neurons per layer. One learning process per each pre-defined architecture is executed. To reduce the computational burden, we restrict the activation function to be identical across all layers, chosen from the following three: $\{\texttt{tanh}, \texttt{ReLU}, \texttt{sigmoid}\}$. To analyze the sensitivity with respect to feature design, we define three feature configurations:

- F_1: 4 features from sequence positions, 4 features from gap constraints, and the subsequence length;
- F_2: F_1 extended with the alphabet size $|\Sigma|$;
- F_3: F_2 extended with the number of sequences m.

After extensive empirical evaluation, the following NN architectures are selected for the RANDOM benchmark:

- $m = 2$: a 3-layer neural network with 10, 5, and 5 neurons at respective layers, apart from one corresponding to the bias; each layer incorporates the activation `ReLU`, with the feature set F_2;
- $m = 3$: a 3-layer neural network with 10, 10, and 5 neurons at respective layers, apart from one corresponding to the bias; each layer incorporates the activation `ReLU`, with the feature set F_2;
- $m = 5$: a 3-layer neural network with 10, 5, and 5 neurons at respective layers, apart from one corresponding to the bias; each layer incorporates the activation `sigmoid`, with the feature set F_1;
- $m = 10$: a 3-layer neural network with 10, 5, and 5 neurons at respective layers, apart from one corresponding to the bias; each layer incorporates the activation `ReLU`, with the feature set F_2.

For training the neural heuristic, we use 32 instances for the training $\mathcal{T}$ and 32 for the validation set $\mathcal{V}$, selecting one instance per each group. For each value of m, this results in 8 instances used for the learning process in the training phase and 8 instances in the validation phase. After the heuristic h_w is learned, the ensemble parameter is set to $\alpha = 0.6$ for $m = 2$, $\alpha = 0.5$ for $m = 3$, $\alpha = 0.35$ for $m = 5$, and $\alpha = 0.15$ for $m = 10$, based on grid search, see Figure 3.

For the REAL benchmark, we employ the same NN architecture as in the $m = 10$ case for the RANDOM benchmark set. This choice allows us to asses the robustness of the learned heuristic on structurally different instances with a larger number of sequences ($m \geq 10$). The IMSBS parameters remain unchanged,

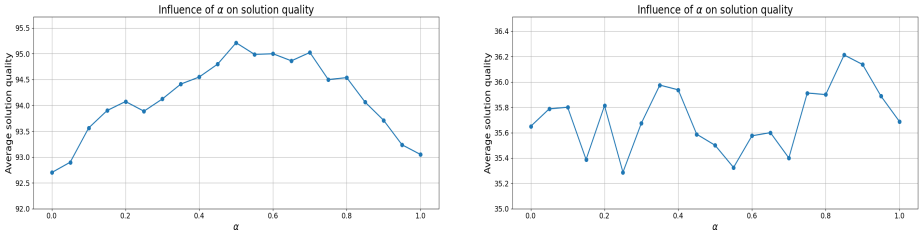

Fig. 3. Visualization of results for tuning parameter α: $m = 3$ (left), and $m = 5$ (right).

Table 1. Comparison of objective values between IMSBS and LIMSBS-ensemble on benchmark set RANDOM. Best results per instance category are displayed in bold.

m	n	$\lvert\Sigma\rvert$	LIMSBS-ensemble	IMSBS
2	50	2	**37.7**	**37.7**
2	50	4	**30.1**	**30.1**
2	100	2	72.4	**72.9**
2	100	4	**62.1**	61.9
2	200	2	**140.7**	139.3
2	200	4	**125.8**	125.5
2	500	2	**299.5**	282.1
2	500	4	**310.9**	303.0
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	100	2	**58.5**	**58.5**
3	100	4	**48.7**	48.5
3	200	2	**106.9**	106.4
3	200	4	**98.0**	97.8
3	500	2	**157.2**	150.6
3	500	4	**238.3**	236.0
Avg.			**115.1**	112.8

m	n	$\lvert\Sigma\rvert$	LIMSBS-ensemble	IMSBS
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	100	2	32.5	**32.9**
5	100	4	27.6	**28.1**
5	200	2	**47.3**	42.1
5	200	4	**41.2**	38.8
5	500	2	**46.0**	39.2
5	500	4	**59.3**	45.6
10	50	2	11.6	**11.7**
10	50	4	**7.5**	**7.5**
10	100	2	**14.4**	12.0
10	100	4	**9.9**	8.9
10	200	2	**14.8**	12.8
10	200	4	**9.5**	8.2
10	500	2	**14.6**	13.2
10	500	4	**10.1**	8.6
Avg.			**23.8**	21.6

and follows those used to test on the set RANDOM, while we set $\alpha = 0.65$ according to grid search performed on benchmark set REAL.

4.3 Results on Benchmark Set RANDOM

Table 1 reports the average results over 10 instances per group for both competitor algorithms. The table is organized into two parts: instance characteristics given by the first three columns followed by two columns reporting the performance metrics of LIMSBS-ENSEMBLE and IMSBS, respectively.

To assess statistical significance, we perform a one-sided (one-sided) Wilcoxon signed-rank test on paired instances (Table 2).

The following conclusions are drawn from these results.

- Out of 32 instances, LIMSBS-ENSEMBLE achieves better performance over IMSBS in 20 cases, ties occur in 8 cases, and in 4 cases the winner is IMSBS.
- The performed statistical evaluation suggests that there is a statistically significant improvement of LIMSBS-ENSEMBLE over IMSBS.

Table 2. Statistical comparison between LIMSBS-ENSEMBLE and IMSBS.

Comparison	Wins / Ties / Losses	p-value	Conclusion
All instances ($m = 2, 3, 5, 10$)	20 / 8 / 4	0.009	Significant improvement
Small instances ($m = 2, 3$)	10 / 5 / 1	< 0.01	Significant improvement
Large instances ($m = 5, 10$)	10 / 3 / 3	< 0.05	Significant improvement

4.4 Results on Benchmark Set REAL

Table 3. Comparison of LIMSBS-ensemble and IMSBS on benchmark set REAL.

RAT instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**26**	69.58	25	30.65	209
15	**12**	40.81	11	24.26	189
20	**13**	16.90	9	49.36	174
25	**7**	24.30	**7**	29.62	173
40	**5**	19.25	4	48.64	154
60	**3**	25.81	1	45.22	154
80	**1**	27.10	**1**	53.02	144
100	**1**	55.98	**1**	44.30	139
150	**1**	56.73	**1**	66.57	131
200	**1**	38.84	**1**	62.54	126

VIRUS instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**84**	163.59	36	43.14	228
15	18	25.21	**19**	36.52	206
20	**12**	24.13	10	26.92	194
25	**12**	25.17	9	30.40	196
40	**7**	23.41	5	32.12	174
60	**5**	25.95	4	34.38	168
80	**4**	34.47	3	40.64	163
100	**3**	55.68	**3**	43.25	160
150	**4**	42.87	3	31.80	157
200	**1**	38.64	**1**	48.59	156

Table 3 summarizes the results, including solution quality and runtime (t in seconds) for both approaches; additionally, reference LCS values from the literature (LCS*) is also reported. The following observations are drawn from these results.

- *Solution quality.* LIMSBS-ENSEMBLE outperforms IMSBS in 12 out of 20 instances, with 7 cases resulting in ties, and only one in favor of IMSBS.
- *Instance difficulty.* The improvements of LIMSBS-ENSEMBLE over IMSBS are more pronounced on challenging instances (e.g., VIRUS datasets), where substantial gains are observed in several occasions. Ties typically occur on more constrained instances (those with a large number of constraints m) with low objective values.
- *Runtime.* The computational effort of both approaches is comparable. While IMSBS is slightly faster on average, the differences are not consistent, and LIMSBS-ENSEMBLE is competitive in most cases.
- *Impact of gap constraints.* Compared to unconstrained LCS values from the literature, the introduction of gap constraints significantly reduces achievable subsequence lengths. This effect is particularly strong for large m (e.g., $m \geq 60$), where feasible solutions become highly restricted, the search space reduced, while the number of root nodes quickly increases.

Overall, the results demonstrate that LIMSBS-ENSEMBLE provides a consistent improvement in solution quality over IMSBS, while maintaining comparable computational efficiency. This makes it a preferable choice when solution quality is the primary objective.

5 Conclusions

This paper addressed the Variable Gapped Longest Common Subsequence Problem (VGLCSP), a challenging extension of the classical LCSP with applications in biological sequence analysis and time-series modeling. We proposed a data-driven learning-based heuristic designed to guide the search process within the state-of-the-art iterative multi-source beam search (IMSBS) framework.

The heuristic is represented by a neural network with a predefined architecture, whose parameters are optimized via a biased random-key genetic algorithm in a neuro-evolutionary setting. To further enhance robustness, we introduced an ensemble heuristic that combines the learned heuristic with the best-performing hand-crafted heuristic from the literature. This hybrid guidance mechanism is integrated into IMSBS to improve search efficiency and solution quality. Experimental results on both synthetic benchmarks and newly introduced biologically inspired instances demonstrate that the proposed ensemble heuristic consistently outperforms the baseline hand-crafted heuristics integrated into IMSBS. In particular, the improvements are most pronounced on challenging instances with complex gap structures, confirming the effectiveness of the learning-guided search.

Future work will focus on extending the approach to protein sequence datasets and larger-scale instances to further assess scalability. Additionally, alternative learning strategies will be explored, including different evolutionary schemes, fitness evaluation criteria beyond mean performance, and improved stopping mechanisms to better control overfitting.

Acknowledgments. This publication is co- funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie COFUND Postdoctoral Programme grant agreement No.101081355– SMASH and by the Republic of Slovenia and the European Union from the European Regional Development Fund. Co-funded by European Union. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

References

1. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
2. Amari, S.i.: Backpropagation and stochastic gradient descent method. Neurocomputing **5**(4-5), 185–196 (1993)
3. Bishop, C.M.: Neural networks and their applications. Review of scientific instruments **65**(6), 1803–1832 (1994)
4. Djukanovic, M., Balaban, N., Blum, C., Kartelj, A., Dzeroski, S., Zebec, Z.: On solving a generalized variable gapped longest common subsequence problem. In: Proceedings of the 20th International Conference on Computer Aided Systems Theory. Springer, 2026. Accepted
5. Djukanovic, M., Raidl, G., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)

6. Djukanović, M., Reixach, J., Nikolikj, A., Eftimov, T., Kartelj, A., Blum, C.: A learning search algorithm for the restricted longest common subsequence problem. Expert Systems with Applications **284**, 127731 (2025)
7. ElSaid, A.A., Ororbia, A.G., Desell, T.J.: The ant swarm neuro-evolution procedure for optimizing recurrent networks. arXiv preprint arXiv:1909.11849 (2019)
8. Galván, E., Mooney, P.: Neuroevolution in deep neural networks: Current trends and future challenges. IEEE Transactions on Artificial Intelligence **2**(6), 476–493 (2021)
9. Gurney, K.: An introduction to neural networks. CRC press (2018)
10. Lafond, M., Lai, W., Liyanage, A., Zhu, B.: The longest subsequence-repeated subsequence problem. In: International Conference on Combinatorial Optimization and Applications. pp. 446–458. Springer (2023)
11. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
12. Lin, G., Chen, Z.Z., Jiang, T., Wen, J.: The longest common subsequence problem for sequences with nested arc annotations. Journal of Computer and System Sciences **65**(3), 465–480 (2002)
13. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: A comprehensive review of architectures, variants, and applications. Information **15**(9), 517 (2024)
14. Ortiz-Bayliss, J.C., Terashima-Marín, H., Conant-Pablos, S.E.: A neuro-evolutionary hyper-heuristic approach for constraint satisfaction problems. Cognitive Computation **8**(3), 429–441 (2016)
15. Peng, Y.H., Yang, C.B.: Finding the gapped longest common subsequence by incremental suffix maximum queries. Information and Computation **237**, 95–100 (2014)
16. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)
17. Pérez Cáceres, L., López-Ibáñez, M., Stützle, T.: An analysis of parameters of irace. In: European Conference on Evolutionary Computation in Combinatorial Optimization. pp. 37–48. Springer (2014)
18. Reixach, J., Blum, C., Djukanović, M., Raidl, G.R.: A neural network based guidance for a brkga: An application to the longest common square subsequence problem. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). pp. 1–15. Springer (2024)
19. SantaLucia Jr, J.: A unified view of polymer, dumbbell, and oligonucleotide dna nearest-neighbor thermodynamics. Proceedings of the National Academy of Sciences **95**(4), 1460–1465 (1998)
20. Watkins Jr, N.E., SantaLucia Jr, J.: Nearest-neighbor thermodynamics of deoxyinosine pairs in dna duplexes. Nucleic acids research **33**(19), 6258–6267 (2005)
21. Wiseman, S., Rush, A.M.: Sequence-to-sequence learning as beam-search optimization. In: Proceedings of the 2016 conference on empirical methods in natural language processing. pp. 1296–1306 (2016)
22. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. IEEE transactions on neural networks and learning systems **33**(9), 5057–5069 (2021)